Arquitectura: Monolito vs Microservicios para TTPN

Fecha: 2026-04-10

Respuesta directa

Mantener el monolito Rails + BD centralizada Supabase, con microapps en el frontend.

No porque los microservicios sean malos, sino porque la naturaleza de los datos de TTPN hace que separarlos sea más costoso que beneficioso. La arquitectura que ya tienes — un solo BE que sirve a múltiples frontends — es exactamente el patrón correcto para tu escala y equipo.

Por qué los microservicios serían un error ahora

1. Tus datos están acoplados por diseño

El trigger `sp_tctb_insert` hace esto en cada travel_count:

```
-- En el mismo INSERT, consulta 1.5M de bookings:
NEW.viaje_encontrado := buscar_booking(vehicle_id, employee_id, ...)
NEW.ttpn_booking_id  := buscar_booking_id(vehicle_id, employee_id, ...)
NEW.payroll_id       := buscar_nomina()
```

Esto funciona en microsegundos porque travel_counts y ttpn_bookings viven **en la misma BD, en la misma transacción**. Con microservicios y BDs separadas, ese matching requeriría:

```
TravelCountService → HTTP → BookingService → respuesta → HTTP →
PayrollService
```

Latencia: de ~2ms (trigger SQL) a ~150–400ms (3 round trips HTTP) por cada captura de viaje.
Y si algún servicio falla a mitad del flujo, tienes un registro inconsistente sin rollback transaccional.

2. El cuadre de nómina y facturación cruza múltiples módulos

```
travel_counts ←→ ttpn_bookings ←→ payrolls ←→ discrepancies ←→ invoices
```

Una "discrepancia PNC" se resuelve cuando un travel_count hace match con un booking. Eso hoy es un `UPDATE` en la misma transacción. En microservicios es un saga pattern con compensaciones — complejidad que tarda meses en implementarse correctamente.

3. Tu equipo no escala para mantener múltiples servicios

Los microservicios requieren:

- CI/CD independiente por servicio
- Monitoreo independiente por servicio
- Versionado de contratos de API entre servicios
- Manejo de fallos parciales (circuit breakers, retries)
- Trazabilidad distribuida (OpenTelemetry, Jaeger)

Un equipo de 2–4 personas pasa más tiempo en infraestructura que en producto.

4. Tu escala no lo justifica aún

Los microservicios resuelven problemas de escala y equipos grandes (Netflix, Amazon). TTPN tiene:

- 54 usuarios concurrentes máximos
- ~30–80 choferes en hora pico
- 1 servidor Railway que maneja todo sin problemas

Shopify corrió como monolito hasta superar $1B en GMV. Basecamp nunca se dividió. Stack Overflow sirve millones de requests diarios con un solo servidor. El monolito no es limitante a tu escala.

Lo que SÍ conviene: Monolito Modular + Microapps

Esta es exactamente la arquitectura que estás construyendo, y es la correcta.

```
┌──────────────────────────────────────────────────────────────┐
│                   MONOLITO MODULAR (Rails API)               │
│                                                              │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│   │ Empleados│  │Vehículos │  │ Viajes   │  │ Nómina   │    │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘    │
│                                                              │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│   │ Clientes │  │ Bookings │  │ Gasolina │  │ Alertas  │    │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘    │
│                                                              │
│  Un solo proceso · Una sola BD · Transacciones ACID          │
└──────────────────────────────────────────────────────────────┘
                              │
                   ┌──────────┼──────────┐
                   ▼          ▼          ▼
          ┌──────────────┐ ┌──────────────┐ ┌──────────────┐
          │  Kumi Admin  │ │  App Móvil   │ │   Portal     │
          │  (Quasar)    │ │  (PWA/Cap.)  │ │   Clientes   │
          │  Netlify     │ │  Netlify     │ │   Netlify    │
          └──────────────┘ └──────────────┘ └──────────────┘
            Administradores   Choferes        Clientes TTPN
```

Microapps = múltiples frontends especializados que consumen el mismo API.
Cada app tiene su propio `registered_app` con módulos permitidos. El backend no sabe ni le importa si

quien llama es un admin, un chofer o un cliente — solo valida el JWT y el scope.

Cuándo sí separar algo del monolito

Hay candidatos válidos para extraer como servicio independiente, pero solo cuando tengan **razón de existir por separado**:

Candidato	¿Cuándo separar?	Tecnología sugerida
N8N	Ya es independiente — automatizaciones que orquestan otros sistemas	N8N en Railway (ya planeado)
Generación de PDFs / reportes	Si el volumen de reportes satura el servidor Rails	Servicio Node.js dedicado en Railway
Notificaciones	Si se necesita Web Push + FCM + email + SMS de forma unificada	Microservicio con Bull/Sidekiq
Procesamiento de imágenes	Si vehicle_checks acumula muchas fotos y el resize satura RAM	Lambda/función en Railway
Portal de Clientes	No es un servicio — es una microapp frontend con `registered_app`	Quasar PWA en Netlify

La regla: **separa cuando el módulo tiene una razón técnica de ser independiente** (escala diferente, tecnología diferente, equipo diferente), no por principio arquitectónico.

Evolución gradual recomendada

Ahora — Monolito + Microapps frontend

```
Rails API (Railway)  →  Supabase Pro
     ↑ ↑ ↑
Kumi Admin · App Móvil · N8N
```

Sin cambios de arquitectura. Foco en producto.

En 1–2 años — Modularizar internamente si el equipo crece

Si el equipo crece a 5–8 devs, empieza a modularizar el monolito con namespaces bien definidos:

```
# app/domains/
#   viajes/         → TravelCount, TtpnBooking
#   nomina/         → Payroll, Discrepancy
#   flota/          → Vehicle, VehicleCheck, GasCharge
#   rrhh/           → Employee, EmployeeMovement
```

Esto permite que cada módulo tenga sus propias reglas de acceso sin romper las transacciones entre módulos.

En 3–5 años — Extraer servicios si el volumen lo justifica

Si `travel_counts` llega a 10M+ registros y las queries de cuadre tardan segundos, ese momento sí justifica evaluar un read replica o un servicio de reportes separado sobre una BD de lectura.

Resumen

Criterio	Microservicios	Monolito modular + Microapps
Consistencia transaccional (trigger buscar_booking)	❌ Muy complejo	✅ Nativo
Equipo pequeño (2–4 devs)	❌ Overhead alto	✅ Enfoque en producto
Múltiples frontends (Admin, Móvil, Clientes)	✅ Aplica	✅ Ya implementado con registered_apps
Escala actual (< 100 usuarios concurrentes)	❌ Innecesario	✅ Más que suficiente
Velocidad de desarrollo de features	❌ Lenta (contratos entre servicios)	✅ Rápida
Costo operativo	❌ Alto (múltiples servicios, logs, CI/CD)	✅ Un solo stack
Evolución futura	Posible migración gradual	✅ Base sólida para escalar

La arquitectura correcta para TTPN es exactamente la que tienes: un BE sólido, una BD centralizada, y múltiples apps frontend especializadas por audiencia.